

Ouvidoria Inteligente: Triagem Semântica de Manifestações Cidadãs

Relatório do desafio prático de NLP Aplicado | [Seu nome] | [Data]

1. Objetivo e visão geral

A ouvidoria de um município de médio porte recebe cerca de 4.000 manifestações por mês e faz a triagem por palavras-chave. Isso deixa passar duplicatas escritas com palavras diferentes ("asfalto esburacado" e "rua com buraco"), não agrupa temas relacionados e perde contexto em textos longos. O projeto constrói um protótipo de triagem semântica sobre 40 manifestações anônimas (5 categorias oficiais, cerca de 15% de duplicatas e 5 textos longos), em quatro entregas: comparação de representações, detecção de duplicatas, chunking e um aplicativo Streamlit.

Todo o código está em um repositório único (**semantic-ombudsman**), gerenciado com **uv**. Os embeddings foram gerados com o modelo multilíngue **paraphrase-multilingual-MiniLM-L12-v2** (sentence-transformers), com vetores normalizados de 384 dimensões, de modo que o produto escalar equivale à similaridade de cosseno.

2. Decisões técnicas por entrega

Entrega 1: BoW, TF-IDF e embeddings

Foram geradas as três representações sobre o mesmo corpus, com *CountVectorizer* e *TfidfVectorizer* usando uma lista de stopwords em português (o scikit-learn só traz stopwords em inglês) e comparação por similaridade de cosseno nos três pares pedidos (M003 x M017, M008 x M022, M008 x M031). Usar a mesma lista de stopwords nas duas representações esparsas manteve a comparação justa. Uma nuvem de palavras construída a partir das frequências do BoW ajudou a inspecionar o vocabulário dominante.

Resultado dos pares (cosseno): **[preencher com a tabela do notebook 01: BoW, TF-IDF e embeddings para os três pares]**. O ponto central é que os pares descrevem o mesmo problema com vocabulário distinto, e por isso as representações esparsas subestimam a semelhança, enquanto os embeddings capturam o significado.

Entrega 2: detecção de duplicatas

A função *detectar_duplicatas(textos, limiar)* gera os embeddings, calcula a matriz de similaridade e percorre apenas o triângulo superior (o índice j começa em $i + 1$), o que elimina a diagonal (um texto comparado a ele mesmo) e os pares repetidos (A,B) e (B,A). O limiar foi escolhido a partir de dados, não por convenção.

Por que não 0.85. Com este modelo, o maior score da base é 0.836 e o limiar de 0.85 não retorna nenhum par. Nas representações esparsas o efeito é ainda mais forte: sem pares até 0.5 no BoW e no TF-IDF, e apenas 5 pares (BoW) e 1 par (TF-IDF, M019 x M031, 0.324) em 0.3. Nos embeddings, a contagem de pares por limiar foi: 0.85 = 0, 0.80 = 2, 0.70 = 15, 0.60 = 65, 0.50 = 131, 0.30 = 430. O salto entre 0.70 e 0.60 indica que, abaixo de 0.70, entram pares que compartilham apenas o tema geral.

Gabarito e métricas. Como o dataset não traz rótulo de duplicata, montei um gabarito manual com 9 pares reais, lendo os textos (buraco na avenida: M003, M017, M038 e M012; praça sem luz: M019 e M031; iluminação: M004 e M019; posto sem atendimento: M008 e M022; lixo em terreno: M006 e M032; falta de remédio: M013 e M034). A tabela compara os limiares:

Limiar	Pares	VP	FP	FN	Precisão	Recall
0.80	2	2	0	7	1.00	0.22
0.75	5	3	2	6	0.60	0.33
0.70	15	6	9	3	0.40	0.67
0.68	26	8	18	1	0.31	0.89
0.65	37	8	29	1	0.22	0.89
0.60	65	8	57	1	0.12	0.89

Adotei **0.68**, o limiar mais alto que ainda alcança o recall máximo da tabela (8 de 9). Abaixo dele o recall não melhora e só crescem os falsos positivos. A escolha prioriza o recall porque, na triagem, um falso positivo custa uma checagem humana a mais, enquanto um falso negativo deixa duas manifestações do mesmo problema tramitando separadas. O custo é a precisão baixa (0.31): muitos pares sugeridos compartilham só o tema (por exemplo, M027 x M036, M036 x M037 e M014 x M033). Uma alternativa de operação em dois níveis seria tratar pares com 0.80 ou mais como duplicata provável (precisão 1.00 nesta base) e os demais como sugestão para revisão.

Falso negativo remanescente. O par M013 x M034 (0.59) descreve a falta de medicamento, mas um é um relato pessoal ("minha mãe precisa retirar um medicamento...") e o outro é genérico. Capturá-lo exigiria um limiar em que o resultado deixa de ser útil. As métricas são otimistas, porque o limiar foi escolhido observando o próprio gabarito, que é pequeno e depende de julgamento.

Entrega 3: chunking

As 5 manifestações mais longas (M012, M018, M027, M036 e M037) foram divididas com o *RecursiveCharacterTextSplitter* em três configurações: A (300/0), B (300/60) e C (150/30). Usei os separadores ["\n\n", "\n", ".", " ", " ", ""] para que o corte preferisse o fim de frase, já que os textos não têm quebras de linha e o padrão do splitter cortaria no meio de frases.

Configuração	Chunks (total)	Similaridade média entre chunks consecutivos
A (300/0)	15	0.5613
B (300/60)	15	0.5613
C (150/30)	31	0.4237

Overlap. As configurações A e B ficaram idênticas: com o separador ". ", o splitter junta frases inteiras e o overlap só reaproveita pedaços já separados. Como cada frase tem cerca de 100 caracteres ou mais, nenhuma cabe em um overlap de 60 e ele não é aplicado. O overlap só apareceu na configuração C, quando o corte caiu no meio de uma frase (em M037, o chunk 4 termina em "caso sejam" e o chunk 5 recomeça em "dos resíduos e, caso sejam"). Ou seja, o overlap funciona como ponte quando o corte cai no meio de uma ideia e é irrelevante quando os cortes coincidem com fronteiras de frase. A menor similaridade média da C se explica pelo tamanho dos chunks, não pelo overlap.

Qual preserva melhor o sentido. A e B. Em M037, os três chunks correspondem a problema (oficina descartando óleo e resíduos), evidências (manchas no chão e galeria de águas pluviais próxima) e solicitação (vistoria e medidas administrativas). Na C, cada frase vira um chunk e o contexto se perde: "O material pode contaminar o solo..." fica sem o referente, e a solicitação final é cortada no meio. Os chunks menores da C, por outro lado, favorecem buscas por detalhes pontuais.

Espaço 2D (PCA). Só M018 forma um grupo compacto (isolada à direita, tema de saúde). M012, M027, M036 e M037 se misturam, e chunks de manifestações diferentes ficam tão próximos quanto chunks da mesma. Uma explicação plausível, não verificada, é que os textos longos seguem um mesmo molde ("a comunidade solicita vistoria..."), e os chunks refletem a função no texto mais do que o tema. Isso é consistente com a Entrega 2, em que essas mesmas manifestações apareciam próximas entre si.

Entrega 4: aplicativo Streamlit

O app (*app_ouvidoria.py*) tem sidebar com seleção do modelo de embedding e top-k configurável, e quatro abas: busca semântica (top-k com score e destaque por cor: verde acima de 0.7, amarelo acima de 0.5 e vermelho nos demais), base completa com botão para o heatmap de similaridade, espaço vetorial em 2D (PCA ou t-SNE, colorido pela categoria oficial) e chunking interativo (colar texto, escolher estratégia e parâmetros, ver chunks e embeddings). O modelo é carregado com *st.cache_resource* e os embeddings com *st.cache_data*, para que a troca de aba não recalcule nada. Observação sobre os clusters do espaço vetorial: **[preencher com o comentário sobre se os clusters coincidem com as categorias oficiais]**.

3. Dificuldades

- **Limiar do enunciado inadequado ao modelo.** O 0.85 não retornou nenhum par. Foi preciso explorar a distribuição de similaridades e calibrar o limiar com base em um gabarito manual.
- **Gabarito subjetivo.** Não há rótulo de duplicata no dataset, e a decisão sobre pares limítrofes (como M004 x M019, que fala de iluminação fraca em vários pontos, contra uma praça sem luz) muda as métricas.
- **Overlap sem efeito visível.** A e B saíram idênticas até eu entender a interação entre o separador de frases e o overlap, o que mostrou que a escolha dos separadores pesa mais que o overlap neste dataset.
- **Detalhes do splitter.** Com o separador ". ", os chunks começaram com um ponto solto. A correção foi usar *keep_separator="end"*.
- **Ambiente.** Diferenças entre o PowerShell e o bash (por exemplo, *mkdir -p*), e um erro de caminho do JSON no app, causado pela posição do arquivo dentro do projeto.

4. Aprendizados

- Representações esparsas dependem de vocabulário compartilhado; embeddings capturam significado, mas também estilo, e por isso textos com o mesmo molde ficam próximos.
- Limiares de similaridade dependem do modelo e dos dados, e devem ser calibrados com um gabarito e discutidos como um trade-off entre precisão e recall, no contexto do uso real.
- A avaliação de qualquer método de detecção exige um conjunto de referência, e o tamanho e a subjetividade desse conjunto limitam a confiança nas métricas.
- Chunking não é só um número: o tamanho, o overlap e os separadores interagem, e cortar em fronteiras naturais de frase preserva mais sentido do que aumentar o overlap.
- Projeções 2D (PCA e t-SNE) servem como indício, não como prova: comprimir 384 dimensões em 2 perde informação, e a leitura dos gráficos deve ser cruzada com métricas.

5. Limitações e próximos passos

O dataset tem apenas 40 manifestações e o gabarito 9 pares, então as métricas indicam tendência e não generalização. Como próximos passos, vale testar outros modelos de embedding (incluindo modelos treinados para português), avaliar um esquema de dois níveis de limiar com revisão humana, agrupar as manifestações por tema (clusterização) para a triagem por categoria e indexar os chunks das manifestações longas na busca semântica, para que a recuperação aponte o trecho relevante e não o texto inteiro.